تعليمات معالج MIPS

د. خولة العلي

MIPS Instruction Set

مجموعة التعليمات الخاصة بالمعالج MIPS هي عبارة عن واجهة (Interface) بين الطبقة الدنيا (Hardware) وبرمجيات النظم.

وهي ثلاث أنواع :

النوع الأول R-Type (Register) : تستخدم للعمليات الحسابية والمنطقية التي تتم بالكامل داخل السجلات ... تتكون من الحقول الآتية :

OP : كود العملية (6 bit) ويكون دائماً 0 في هذا النوع

rs &rt : السجلات المصدر (5 bit كل منهما)

rd : السجل الهدف الذي تخزن فيه النتيجة 5 bit

Shmat : مقدار الإزاحة ،يستخدم في تعليمات الإزاحة

Func : كود الوظيفة ، يحدد نوع العملية الدقيق (مثل الجمع أو الطرح) لأن حقل op يكون صفراً

كل المعطيات (Arguments) عبارة عن مسجلات (Registers) فقط، أي تتم العمليات بين المسجلات مثال جمع رقمين موجودين في سجلين : مثال :

add $t0, $t1, $t2

أي :

$t0 = $t1 + $t2

النوع الثاني I-Type (Immediate) :

تتكون من الحقول التالية :

op: كود العملية

rs:السجل المصدر (أو سجل القاعدة في حالة الذاكرة)

rt: السجل الهدف (أو السجل الذي سيتم التحميل إليه/التخزين منه)

Immediate : قيمة فورية طولها 16 bit (ثابت أو إزاحة عنوان)

تتعامل مع مسجل (Register) وأرقام مباشرة (Immediate values) أو عناوين. مثل:

Addi $t0, $t1, 5

أي:

$t0 = $t1 + 5

النوع الثالث J-Type (Jump) :

تستخدم للقفز غير المشروط وتتكون من كود العملية 6 bit وعنوان طوله 26 bitلمكان معين في البرنامج، وتتعامل مع عناوين (Address) . **مثل:**

j 1000

أي :

تنفّذ التعليمات من العنوان 1000

ملاحظة : عدد العمليات الممكنة في MIPS هي : $2^6=64$

Instruction Fields

أي تعليمة في MIPS تنقسم لعدة أجزاء :

1. **كود العملية (Opcode) :** حجمه(6bits) ، يحدد نوع العملية الفعلي (جمع، طرح، قفز، إلخ).
2. **المعاملات (Operands) :**كل معامل حجمه(5bits) ، يحدد مصدر القيم (Source) ووجهتها(Destination) .
3. القيم المباشرة (Immediate values):أرقام مدمجة داخل التعليمة نفسها أحجامها متغيرة (bit26,bit16,bit5) وحسب التعليمة إما تعتبر موجبة دوماً أو (موجبة/سالبة).

وعندما نتعامل مع تعليمات حسابية أو تعليمات مقارنة مثل:

addi $t0, $t1, 3

وتعتبر موجبة، ولو وضعنا (-3) تعتبر سالبة.

أما عندما نتعامل مع تعليمات المنطق مثل : "andi$t0, $t1, 5 " هنا تعتبر دوماً موجبة ومن الخطأ وضعها سالبة.

MIPS Instruction Set

R-Type : تحتوي التعليمة على 3 معاملات مسجلات :(Register Operands) اثنان كمصدر (2 Sources) وواحد كوجهة (1 Destination) .

I-Type : تحتوي على معاملين مسجلين (2 Register Operands) وقيمة ثابتة مباشرة (Immediate constant) يمثلها 16bit الأرقام المباشرة أحجامها متغيرة، وحسب التعليمة إما تعتبر موجبة دوماً أو (موجبة/سالبة).

J-Type : تحتوي على قيمة مباشرة (Immediate) تمثل بـ 26bit .

R	op	rs	rt	rd	shamt	funct
	6bits	5bits	5bits	5bits	5bits	6bits

I	Op	Rs	Rt	immediate
	6bits	5bits	5bits	16bit

J	Op	address
	6bit	26bit

فائدة اربيسيزية: لماذا المعالج في النوع (R-Type) يستخدم حقل funct لتحديد العملية وليس الـ opcode؟ لأن (R-Type) يحتوي على عدد كبير من العمليات وجميعها لها نفس الـ Opcode (والذي يساوي 0). فاستخدموا حقل funct للتمييز بينها. أما في الـ (I-Type) والـ (J-Type) فعدد العمليات أقل ويوجد مساحة كافية في الـ Opcode لتمييزها مباشرة ، وكل تعليمة لها opcode خاص بها.

لماذا المعالج يتعامل مع الـregisters ولا يتعامل مع الـRAM ؟

الـRegisters وجدت لرفع الأداء ضمن الـcpu لأنها قريبة من المعالج وحجمها صغير ،...

معلومة جانبية : ((يوجد داخل المعالج 32 Register وكل register سعته 32 بت وبالتالي فإن إجمالي سعة التخزين في الريجسترات = 32 * 32 = 1024 فأثناء التنفيذ يتم نقل التعليمة مع معاملاتها إلى الريجسترات بشكل مؤقت ليتم تنفيذها))

وعدد الـcycle في الـRAM التي ستستهلكها في القراءة أكثر منها في الـRegisters

وأضيف حل آخر من أجل السرعة وذلك إضافة ذاكرة الـcach التي يخزن فيها جزء من البرنامج ليتم تنفيذه ، ثم الجزء الآخر وهكذا ...

وهناك حلول أخرى مثل تقنية التوارد pipeline

- يتم تخزين البيانات البسيطة Simple variables التي تحتوي على قيمة واحدة داخل registers وذلك لأنها سريعة ويمكن للمعالج التعامل معها
- أما البيانات المركبة ، مثل : Arrays , structures فيتم تخزينها في الـmain memory (RAM) لأن حجمها كبير ولا يمكن استيعابها داخل عدد الريسجترات المحدد.
- تتم جميع العمليات الحسابية Arithmetic operations (مثل الجمع والطرح) فقط على القيم الموجودة داخل الـregisters ولا يمكن تنفيذ هذه العمليات مباشرةً على البيانات الموجودة في الذاكرة يجب تحميلها أولاً إلى الريجسترات باستخدام تعليمات مثل load ثم إجراء العملية الحسابية وبعد ذلك يمكن تخزين النتيجة مرة أخرى في الذاكرة باستخدام تعليمة store
- يمكن للذاكرة الرئيسية تخزين كميات كبيرة جداً من البيانات (عدة جيجابايتات) تصل للمليارات مثال على ذلك بنية MIPS

↙ التعامل مع الذاكرة والمسجلات(Registers vs Main Memory)

تعليمات تنقل البيانات بين الذاكرة والسجلات نعلم أن التعليمات الحسابية تتعامل فقط مع الريجسترات لذلك نحتاج تعليمات لنقل البيانات بين الريجيسترات والذاكرة ومن هذه التعليمات هي تعليمة

❖ Load (LW)

- تنسخ البيانات من الذاكرة وتضعها في السجل:

مثل : LW $t0, 4($t1)

- أجلب القيمة الموجودة في العنوان ($t1 + 4) وضعها في.$t0

ملاحظة ($t1 + 4): هنا أخذنا عنوان t1 وليس قيمته.

❖ Store (SW) :

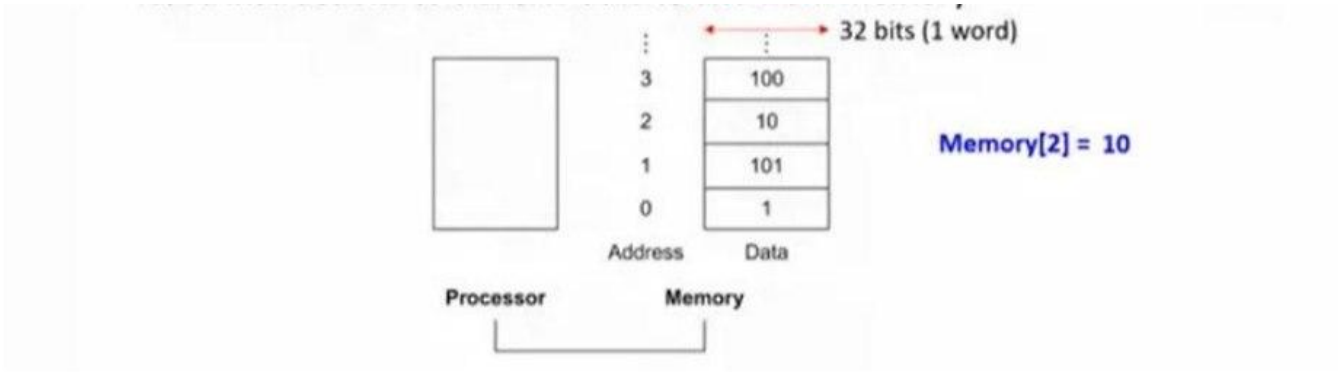
- تنسخ البيانات من السجل إلى الذاكرة:

مثل: SW $t0, 16($t1)

- خزّن قيمة $t0 في العنوان.($t1 + 16)

Memory Operands: Byte Addressing

تعتمد معمارية MIPS على Byte Addressing كل عنوان في الذاكرة يُشير إلى Byte واحد أي 8 bit أي أن كل موقع في الذاكرة يشير إلى بايت وحد فقط وليس إلى كلمة word كاملة



الكلمة (Word) في MIPS تشغل 4 عناوين يعني 32bitأي 4byte وبالتالي الكلمة الواحدة تشغل 4عناوين في الذاكرة لذلك فإن عناوين الكلمات المتتالية تختلف بمقدار 4 byte مثال إذا كان عنوان كلمة معينة في الذاكرة (0) فإن الكلمة المتتالية سيكون عنوانها 4 ثم 8 ثم 12 .... .

في لغة MIPS من أجل الوصول لعنصر معين في المصفوفة A[index] يجب ضرب (Index * 4) لحساب العنوان بشكل صحيح (لأن العنصر A[index] يمثل word وبين كل word وأخرى Byte4 لذلك يتم ضرب الفهرس المصفوفة بالعدد (4)

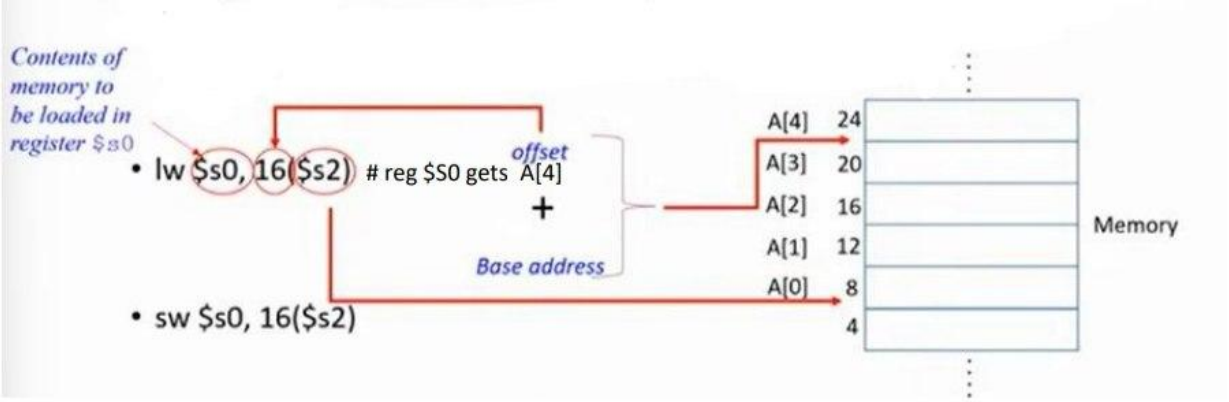
والناتج يمثل قيمة الإزاحة عن العنوان القاعدي فمن أجل الوصول للعنوان نجمع قيمة الإزاحة مع العنوان القاعدي

مثال(1): نريد تحميل القيمة الموجودة في A[4] في السجل $S0 :

lw $S0, 16($S2) # Reg $S0 gets A[4]

يحتوي على القيمة التي تم تحميلها من الذاكرة

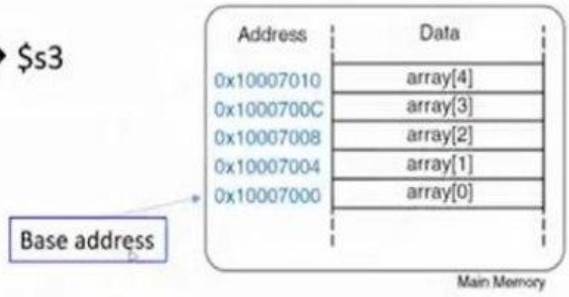
قيمة الإزاحة : بما أن A[4] ... 24=16+8=($s2)+4*4 أي أن عنوان العنصر A[4] هو 24



مثال (2):

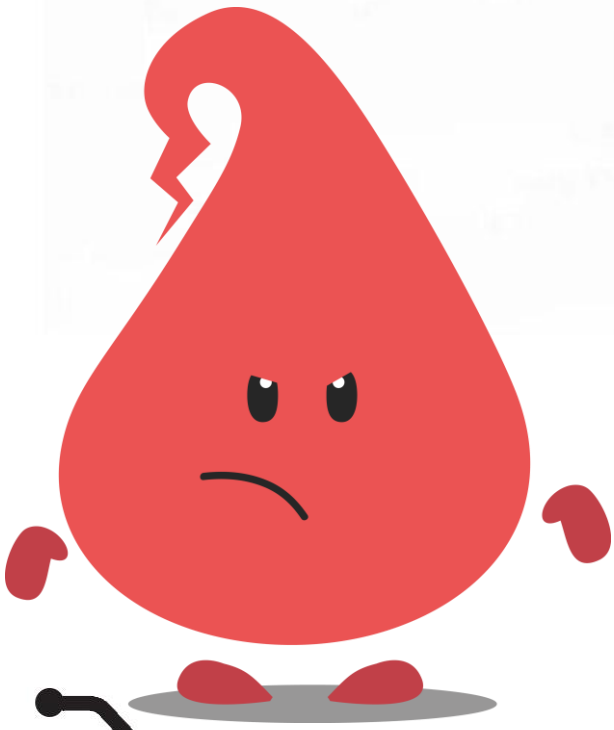
• Compilation example:
  • A[12] = h + A[8];
  • Variable h → $s2, base address of Array A → $s3

For more illustration



For example:
A[4] is at address = $(4x4)_{10}$ + 0x10007000
= 0x00000010+0x10007000
= 0x10007010

Note: 0x is a notation for hexadecimal form



وَمَن يَتهَيّب صُعُودَ الجِبَالِ
يَعِش أَبَدَ الدَهرِ بَينَ الحُفَر

تمرين: لدينا التعليمة التالية لغة عالية المستوى :

مثلها كيف تصبح بلغة الـAssembly :

A[12] = h + A[8];

Variables h: $s2 , base address of array A : $s3

يقوم الـcompiler بتحويل هذه التعليمة إلى :

أولاً: تحميل قيمة A[8] في سجل وليكن $t0

مقدار الإزاحة : 8×4 lw $t0 , 32($s3)

ثانياً: بجمع السجل $t0 (الذي يحوي قيمة العنصر A[8])مع السجل $s2 (الذي يحوي قيمة h)

add $t0 , $s2 , $t0

ثالثاً: يخزن قيمة الجمع ($t0) في الذاكرة عند عنوان العنصر A[12]

حساب عنوان A[12] = base address + مقدار الإزاحة

$$12 \times 4 = base\ address$$

sw $t0 , 48($s3)

MSB (Most Significant Byte): البايت الأكبر أهمية ← يسار العدد.

LSB (Least Significant Byte): البايت الأقل أهمية ← يمين العدد.

Memory Operands (Byte Order) :

هناك طريقتان لتنظيم البايتات داخل الكلمة الواحدة:

الأكبر أخيراً: (Big-endian) يبدأ ترقيم البايتات من الطرف الأكبر، حيث يتم تخزين البايت الأكثر أهمية (MSB) في العنوان الأصغر (اليسار)

Byte #	0	1	2	3
القيمة(0x23456789)	23	45	67	89
	MSB			LSB

الأصغر أخيرا: (Little-endian) يبدأ ترقيم البايتات من الطرف الأصغر، حيث يتم تخزين البايت الأقل أهمية (LSB) في العنوان الأصغر (أول بايت في الذاكرة) (اليمين)

Byte #	3	2	1	0
القيمة(0x23456789)	23	45	67	89
	MSB			LSB

Word Address عنوان الكلمة ثابت ولا يتغير , بتغير النظام لكن تراتيب البايتات هو الذي يختلف .

تمرين :

بفرض لدينا 0x2345678 = $S0 وتم تنفيذ التعليمات التالية :

sw $S0 , 0($0)

lb $S0,1($0)

المطلوب : ما قيمة الجديدة ل $0 بالنظام Big-endian و Little-endian ؟

أولا : في نظام ال Big-endian :

sw $S0,0($0)

أي: خزن القيمة الموجودة في السجل $S0 داخل العنوان التالي offset + base.addres

((السجل $0 لا يملك عنوان , لكن قيمته (0) تستخدم لحساب العناوين)) 0+0=0

بما أن النظام Big-endian فيتم تخزين البايت الأكثر أهمية MSB في أصغر عنوان وهو(0)

0	1	2	3
23	45	67	89

Lb $S0,1($0)

أي يتم تحميل القيمة الموجودة في العنوان 1+0=1 والتي هي (0x45) داخل السجل $S0 ، و لأن التعليمة lb تعمل (التوسيع) sign extension فتصبح القيمة $s0=0x0000045

ثانيا : في نظام Little-endian :

SW $S0,0($0)

يتم تخزين قيمة $S0 في الذاكرة عند عنوان (0) , والبايت الأقل أهمية LSB يخزن في العنوان (0) بالشكل التالي :

3	2	1	0
23	45	67	89

Lb $S0, 1($0)

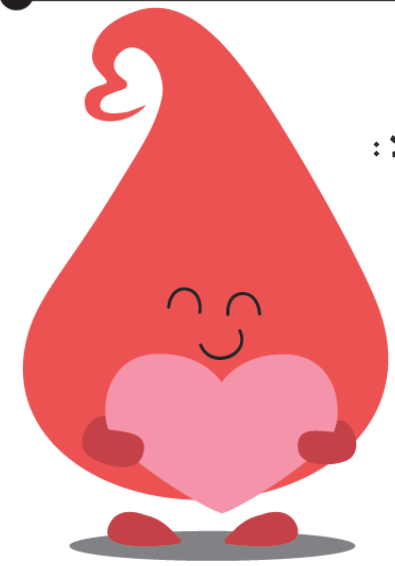
$s0 = 0x00000067

Constant / Immediate Operands – Design Principles

في كثير من الحالات نستعمل الثوابت. للحصول عليها بدون الذاكرة .

مثال في لغة عالية المستوى , لزيارة فهرس مصفوفة :

```
N=0;
A[N]=L*2;
N=N+1;
A[N]=l*3;
```



مثال في لغة Assembly ,لإضافة الثابت (4) الى السجل $S3 نحتاج الى تحميل الثابت أولا :

```
lw $t0 ,addi Constant 4($S1)
Add $S3 ,$S3,$t0
```

لتجنب تعليمة التحميل , يوفر MIPS تعليمة الجمع السريع add Immediate او addi .

لا يوجد وصول لذاكرة وبالتالي أسرع بكثير ويستهلك طاقة أقل .

لا يوجد تعليمة Subi بل نستخدم addi مع ثابت سالب .

Review of Binary Numbers – Unsigned Numbers

تمثل الحواسيب الأعداد بأرقام ثنائية أو بتات (Base 2) أي (0,1)

لحساب قيمة عدد في أي أساس باستخدام قانون :

$$\sum_{i=0}^{n-1} di \times Base^{i}$$

مثال (1011) الى Decimal :

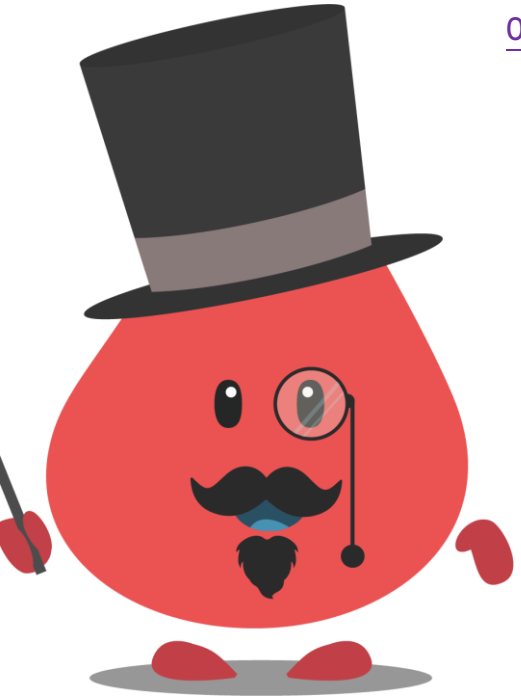
$(1\times2^3) + (0\times2^2) + (1\times2^1) + (1\times2^0)$

$= (1\times8) + (0\times4) + (1\times2) + (1\times1)$

$= 8 + 0 + 2 + 1 = 11$ (ten)

طرق تمثيل الأعداد الموقعة Signed Number Representations

1. Sign and Mangnitude Representation



نفصل الإشارة عن القيمة ، يتم تمثيل العدد باستخدام بت الإشارة MSB مثال: 0111

بت الإشارة MSB =0 ⇐ العدد موجب

باقي البتات (111) تمثل القيمة ⇐ 7

لَيسَ الجَمالُ بِأَثوابٍ تُزَيِّنُنا

إِنَّ الجَمالَ جَمالُ العِلمِ وَالأَدَبِ

وَلَيسَ اليَتيمُ مَلا والِدَينِ لَهُ

إِنَّ اليَتيمَ يَتيمُ العِلمِ وَالأَدَبِ

مثال 1111 :

MSB =1 ⇐ العدد سالب

111 ⇐ 7 ⇐ الجواب : 7-

مشاكل هذا التمثيل:

يعاني هذا التمثيل من عدة مشاكل أهمها :

1- وجود تمثيلين للصفر ، حيث يمكن تمثيل صفر موجب (+0) و صفر سالب (-0) وهذا غير منطقي ويسبب تعقيداً في العمليات

2- الحاجة إلى معالجة خاصة لبت الإشارة حيث يتم فصل الإشارة عن القيمة ، مما يزيد من تعقيد تصميم الدارات.

2. One's complete representation

يتم تمثيل العدد السالب بعكس جميع البتات (0↔1) ، مثال:

+7=0111

-7=1000 ⇐ بالمتمم الأحادي

مشاكل هذا التمثيل :

1- وجود تمثيلين للصفر (+0 / -0) مما يسبب الفيض وعدم الكفاءة في التمثيل

المتمم الثنائي (Two's Complement)

- الطريقة المعتمدة لتمثيل الأعداد الموقعة في الأنظمة الحاسوبية، في هذا التمثيل لا يتم فصل الإشارة عن القيمة ، بل يمثل العدد بالكامل بشكل ثنائي واحد .
- يتم تحديد إشارة العدد من خلال البت الأكثر أهمية MSB حيث إذا كان MSB=0 يكون العدد موجباً وإذا كان MSB =1 يكون سالباً

لتمثيل عدد سالب Tow's complement يتم اتباع الخطوات التالية :

1. كتابة القيمة المطلقة للعدد الثنائي
2. قلب جميع البتات (0↔1)
3. إضافة 1 للناتج

تحويل العدد من tow's complete إلى decimal :

يوجد طريقتين:

1- الطريقة المباشرة باستخدام القانون :

$$d_{n-1} \times -2^{n-1} + \sum_{i=0}^{n-2} di \times 2^{i}$$

مثال:

$$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 0000_{(2)}$$
$$(1 \times -2^{31}) + (1 \times 2^{30}) + (1 \times 2^{29}) + \cdots + (0 \times 2^{1}) + (0 \times 2^{1}) + (0 \times 2^{0})$$
$$= -2^{31} + 2^{30} + 2^{29} + \cdots + 0 + 0$$
$$= -16_{(10)}$$

في حال كان العدد سالباً:

1- نحول العدد إلى موجب باستخدام tow's complete

2- نحسب قيمته العشرية (باستخدام القانون) إضافة الإشارة السالبة للنتيجة

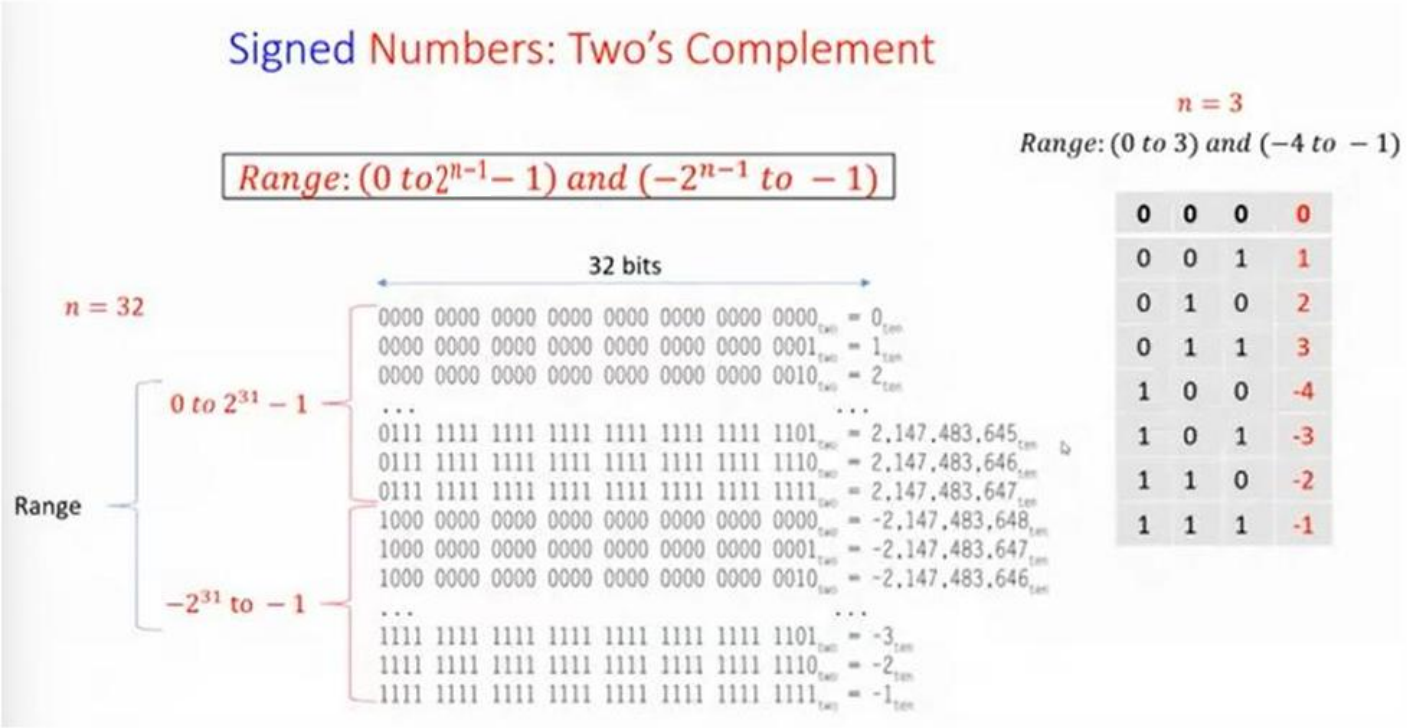
مثال:

$$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 0000$$
$$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001\ 0000$$
$$(0 \times -2^{31}) + (0 \times 2^{30}) + \cdots + (1 \times 2^{4}) + (0 \times 2^{3}) + (0 \times 2^{2}) + (0 \times 2^{1})$$
$$= 0 + 0 + 0 + \cdots + 16 + 0 + 0 + 0 + 0$$
$$= -16_{(10)}$$

نعكس إشارة النتيجة ⇐ 16-

تستخدم هذه القاعدة في تحويل أي عدد ثنائي موقّع إلى قيمته العشرية بسهولة

✓ مدى الأعداد الموقعة :



✓ الأعداد غير الموقعة :

- جميع البتات تستخدم لتمثيل القيمة فقط ولا يوجد بت مخصص للإشارة
- بت مخصص للإشارة
- عدد القيم الممكنة : (بما أن لدينا 32 bit) $2^{32} = 4{,}294{,}967$
- يتم ترقيم البتات من اليمين لليسار

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

MSB (32 bits wide) LSB

$$Range(0\ to\ 2^{32} - 1) \rightarrow (0 \rightarrow 4{,}294{,}967{,}295)$$

يحدث overflow عندما نحاول تخزين قيمة أكبر من الحد الأقصى

Sign Extension

يستخدم عند تحويل عدد من عدد بتات صغير (bits16) الى عدد كبير مثلا (bits32)

نكرر البت الأكثر أهمية MSB عند التوسيع اذا العدد سالب (MSB=1) نضيف واحدات (1)

اذا العدد موجب (MSB=0) نضيف لصفار (0)

مثال: العدد:(+2)

0000 0000 0000 0010 → (two)

0000 0000 0000 0000 0000 0000 0000 0010

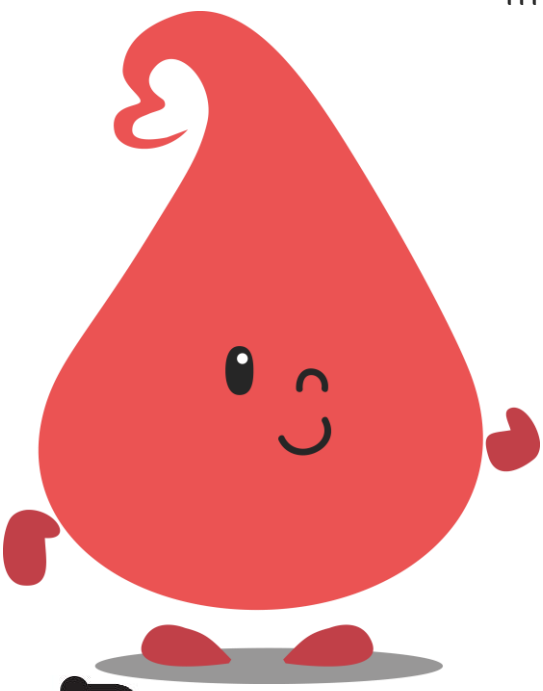
مثال : العدد:(-2)

1111 1111 1111 1110 → (two)

1111 1111 1111 1111 1111 1111 1111 1110

(two)

يحافظ على نصف القيمة بعد زيادة عدد الخانات



Success is not final; failure is not fatal: It is the courage to continue that counts

تمثيل التعليمات في الحاسوب

التعليمات داخل الحاسوب هي عبارة عن إشارات إلكترونية عالية (1)ومنخفضة (0) المستوى، وبالتالي يتم تمثيلها كأرقام

يوجد بعض المصطلحات الأساسية في تمثيل التعليمات:

-الحقل :(Field) هو جزء من التعليمة المُمثَّلة يحتوي على معلومة معينة.

-كل حقل يحتوي على عدد معين من bits.

-تنسيق التعليمة :(Instruction Format) هو عبارة عن مخطط أو الترتيب الذي تتوزع فيه الحقول.

-حيث كل تعليمات MIPS هي عبارة عن 32 bits تتوزع بين الحقول.

▪لغة الآلة:(Machine Language)

-تُعبّر عن المستوى الرقمية من التعليمات، ويتم تسميتها بمجموعة التعليمات للطبقة المتسلسلة (Machine code).

-حيث أن تعليمة MIPS تُمثّل data من 32 bits من أصفار وواحدات داوماً بلغة الآلة.

كيف تُنسَّق التعليمة في bit32؟؟

في MIPS يوجد ثلاثة أنواع من التنسيقات، وكل تنسيق يختلف من حيث عدد الحقول وطريقة تمثيل bits32

النوع الأولR-type : ▪يعتمد على السجلات (Registers) كمعاملات، حيث يحتوي عادةً على 3 معاملات من السجلات، وسجلان للمصدر وواحد للوجهة لتخزين النتيجة.

6 bit	5 bit	5 bit	5 bit	5 bit	6 bit
op	rs	rt	rd	shamt	funct

opcode، وقيمته (0) في التعليمة من نوع R-type.

-هو الذي يحدد تنسيق التعليمة.

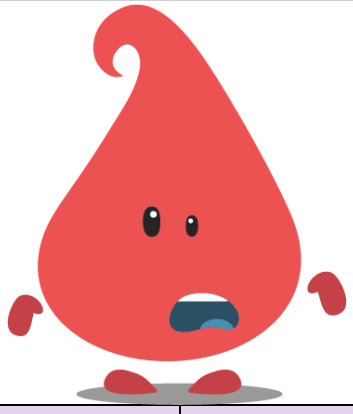
rs : سجل المصدر الأول.

rt : سجل المصدر الثاني.

rd: سجل الوجهة الذي سيتلقى نتيجة العملية.

shamt: كمية الإزاحة (تُستخدم فقط في تعليمات الإزاحة، وبخلاف ذلك تكون صفراً)

Funct (function) : لترميز التعليمة الفعلية ، يستخدم للتمييز بين العمليات المختلفة التي تشترك بنفس الـ opcode مثل add , sub



مثال: add $t1, $s1, $s2 :

كيف تُنسَّق؟

- نُلاحظ أنها تجمع بين السجلات فقط من نوع R-type.
- حيث أن كل سجل يمتلك قيمة عددية فعلية.
- المعطيات $t1 = 9 :، $s1 = 17، $s2 = 18
- ← تنسيق التعليمة بالشكل(add $t1, $s1, $s2) :

op	rs	rt	rd	shamt	funct
0	17	18	9	0	32

- الآن قمنا بتحويل رموز التعليمة إلى أرقام، حيث كل رمز تم حسابه بالقيمة العددية المخصصة له.
- أيضاً تم تحويل كل سجل إلى رقمه الخاص.
- ثم تم توزيع القيم العددية على الحقول.

op (6bit)	rs (5bit)	rt (5bit)	rd (5bit)	shamt (5bit)	funct (6bit)
000000	10001	10010	01001	0000	100000

- ثم من هنا تم تحويل كل رقم عشري في الحقول إلى قيمة ثنائية.
- فأصبح لدينا Word من 32 bits [سلسلة من الأصفار والواحدات] وتكوّن النسخة الرقمية التي يتم تخزينها في الذاكرة لينفذها المعالج بعد ذلك.

-في المثال السابق: تعرّفنا كيف تتم عملية الترجمة من لغة التجميع إلى لغة الآلة [تحويل الرموز البرمجية إلى سلسلة من الأرقام الثنائية (0) و (1) بطول 32 bits] من خلال مراحل دقيقة:

1.تحديد نوع تنسيق التعليمة[R-type or I-type or J-type]

2.تحويل الرموز البرمجية إلى قيم عددية مثلاً:

-تحويل كل رمز سجل إلى القيمة الخاصة به.

-تحديد الرقم الخاص بالـ(opcode الرقم الخاص بنوع العملية)

-مثال opcode :الخاص بتعليمات0 = ALU

3.توزيع القيم على حقول التعليمة – مثل ما عملنا في المثال السابق في الـ①

4.التحويل إلى النظام الثنائي – في المرحلة ② في المثال السابق

مع الانتباه: عند التحويل من الأرقام العشرية إلى الثنائية يجب الالتزام بعدد الخانات (bits) لكل حقل لضمان بقاء طول التعليمة الكلي(bits32)

- سؤال: بعد تعلّمنا كيفية تنسيق تعليمة من نوعR-type ، هل من الجيد من استخدام نفس التنسيق لتعليمة من نوعI-type ؟ولماذا؟

- ليس من الجيد: أولاً تعليمة من نوع I-type تتعامل مع قيم immediate(ثابتة) أو عنوان] offset يعني أن التعليمة تحتوي على قيمة ثابتة مُخزّنة بداخلها وليس داخل سجل]

```
Addi  $t1, $t0, 3
lw    $t2, 4($s0)
```

- لأنه تنسيق R-type كل حقل منه يُغطي فقط 5] bit أي أكبر قيمة يمكن تمثيلها من الأقل 2^5 = 32.[
- وبوجود ذلك كمعمارية MIPS التي تكون فقط 32 سجل، وكل سجل يمتلك 32.bit
- ميزات لو استخدمنا تنسيق R-type لتعليمة من نوع I-type فيجب مصدر constant و offset في حقل 5 bits هذا النطاق أي نطاق [-15 , +16] السبب: نخزن أعداد مع إشارة فالـ5 بتات منها بت للإشارة, فأكبر عدد يخزن ب 4 بتات هو 15 واصغر عدد هو -16
- هذا النطاق صغير جداً ويمكن فقط الوصول لعناصر محدودة جداً داخل الذاكرة أو هيكلية البيانات ... لحل هذا الصراع بين الحفاظ على نفس طول جميع التعليمات (32 bit) والحاجة إلى المرونة (جعل كل نوع من التعلميات تنسيقه الخاص) وهو ما أدى لمبدأ أساسي من مبادئ التصميم

Good design demands good compromises

أي: التصميم المعماري هو نتاج توازن [تسوية] بين التنظيم والثبات والوظيفية.

يكون التوازن من خلال:

- الحفاظ على جميع التعليمات بطول 32 bits واحد.
- استخدام أنماط مُختلفة من التنسيق حسب نوع التعليمة.

مثال:

← R-type – التعامل مع السجلات فقط.

← I-type – التعامل مع ثوابت/تحميل من الذاكرة.

النوع الثاني(Immediate) I-type :

6 bit	5 bit	5 bit	16 bit
op	rs	rt	constant / Address

- الحقل ذات 16 bits للتعليمة :lw لتحميل كلمة(Word) ذات النطاق ما بين $byte(32,768)$ أو[-2^{15} , $+2^{15}$] من العنوان من السجل الأساسي rs
- الإضافة الفورية :(add immediate) (addi) لا يُسمح لها باستخدام ثوابت لا تتجاوز ⊛ 2^{15}.

مثال LW :بتنسيقI-type :

lw $t0, 32($s3)

- ▪تعليمة لتحميل كلمة واحدة من الذاكرة ذات العنوان (محتوى سجل $s3+32)
- تعامل السجل $s3 وكسجل أساسي يُمثّل بداية جزء من الذاكرة.
- السجل $t0 يُمثّل سجل الوجهة الذي سيُخزّن القيمة بداخله.

كيف تُنسَّق؟

op (6bit)	rs (5bit)	rt (5bit)	constant/Address (16bit)
	19	8	32

- حيث$t0 = 8 :، $s3 = 19

أنه لكل سجل قيمة عددية فعلية :

$s0 – $s7: 16 – 23
$t0 – $t7: 8 – 15
$t8 – $t9: 24 – 25

تعود أهمية هذه القيم إلى أن:

السجلات في MIPS ليست مجرد رموز أو أسماء، بل هي مواقع صغيرة داخل الـ CPU تُخزَّن فيها قيم حقيقية لأرقام/نتائج جمع/طرح – عناوين ذاكرة...

–والعتاد (Hardware) يحتاج مكان سريع لحفظ البيانات أثناء التنفيذ في كل تعليمة مثل:

(addi, sw, add, lw) يجبر cpu على قراءة البيانات من مكان ما، فإن ملف السجلات Registers هو المكان السريع (داخل المعالج) والذي يتكون من 32 سجل وكل سجل من bit32
بحيث من خلال هذه القيم، يمكن لـ (ALU ذاكرة، وحدة التحكم) أن تقرأ هذه القيم وتكتب إليها مباشرة في كل دورة ساعة, بدلا من الذهاب الى الذاكرة البطيئة في كل مرة

مثال : ليكن لدينا تعليمة بلغة C

Register Filesهو الجانب السريع داخل العمالج الذي يتكون من 32 سجل وكل سجل من bit.32
من خلال هذه القيم يمكن لـ ,ALU) ذاكرة، وحدة التحكم (أن تقرأ هذه القيم وتكتب إليها مباشرةً في كل دورة ساعة، بدلاً من الذهاب إلى الذاكرة البطيئة في كل مرة.

C code: A[300] = h + A[300]

حيث $t1 = :السجل الأساسي الذي يُمثّل بداية المصفوفة.

شحن السجل بنتيجة جمع [A[300 + h#

add $t0 , $s2 , $t0

تخزين نتيجة جمع [A[300 + h# في [A[300

sw $t0 , 1200($t0)

◀ بشكل تنسيق التعليمة

	op	rs	rt	rd	shamt	funct
lw	35	9	8	1200		
add	0	18	8	8	0	32
sw	43	9	8	1200		

← ▪تحويل من الأرقام العشرية إلى ثنائية:

100011	01001	01000	0000 0100 1011 0000		
0000000	10010	01000	01000	00000	100000
101011	01001	01000	0000 0100 1011 0000		

مثال : لتكن لدينا كود بلغةC

f = (g + h) + (i + j) + (k + 1) ;

نريد تمثيل الكود بلغة:Assembly

ستعتبر كل رمز هو عبارة عن سجل :

f → $s0, g → $s1, h → $s2, i → $s3

j → $s4, k → $s5, l → $s6

MIPS code:

```
add  $t0, $s1, $s2   # (g + h)
add  $t1, $s3, $s4   # (i + j)
add  $t2, $s5, $s6   # (k + L)
add  $t3, $t0, $t1   # (g+h) + (i+j)
add  $s0, $t3, $t2   # f = (g+h) + (i+j) + (k+L)
```

المكافئ لها من حيث القيم العددية للسجلات:

Add $8 , $17 , $18

Add $9 , $19 , $20

Add $10 , $21 , $22

Add $11 , $8 , $9

Add $16 , $11 , $10

عمليات الضرب والقسمة فيMIPS

الضرب:

الفكرة الأساسية: أن ضرب عددين 32 bit قد ينتج ناتج bit64 ، مما يعني أن النتيجة قد لا يمكن وضعها كلها في سجل واحد من bit.32

الصيغة العامة لتعليمة الضرب:

mult register1, register2

نتيجة الضرب لا تذهب مباشرةً إلى أي سجل عادي (لأنه من المحتمل أن يكون ناتج الضرب أكبر من محتوى السجل (32 bit) ، لذلك تُخزَّن في سجلين خاصين:

سجل :HI يُخزَّن الـ 32 bit العليا من الناتج.

سجل :LO يُخزَّن الـ 32 bit الدنيا من الناتج.

يتم التقسيم الناتج بهذه الطريقة لأن hardware يحتاج طريقة لحفظ النتيجة كاملة بدون فقدات أي bit عندما يكون الناتج أكبر من سعة سجل واحد.

مثال:

C: a = b * c

in MIPS:

- نمثل الرموز بسجلات:

a → $s0, b → $s2, c → $s3

- أضف السجل ($s1) إذا كان ناتج الضرب أكبر bit:32

```
Mult  $s2, $s3   # b * c
#     mfhi  $s0   تم تخزين نصف الـbitsمن الـ bits العليا في$s0
#     mflo  $s1   تم تخزين نصف الـ bits الدنيا في$s1
```

إذا لم يشك شخص واحد أسبوعياً بأنك مجنون فأنت لا تحدث تأثيراً حقيقياً في هذا العالم

القسمة:(Division)

الصيغة العامة لها:

div register1, register2

هذه العملية تقسّم قيمة السجل الأول على قيمة السجل الثاني، ونتيجة القسمة كحفظ أيضاً في أكثر من سجل:

سجل :LO يحتوي على ناتج القسمة الخارج.(quotient)

سجل :HI يحتوي على الباقي.(remainder)

استخدام العمليات المنطقية فيMIPS

- تقعد على التعامل مع البيانات على مستوى bit لا.(Bit-level) bit
- وهي ضرورية جداً لاستخراج أو إدخال مجموعات من Bits داخل الكلمة الكاملة.

القناع:(Masking)

- يُستخدم القناع لعزل Bits أو Bytes أو معينة داخل الكلمة مع كتما هل القيمة.
- تقعد هذه العملية على منطق:[bit by bit] AND
- عند إجراء عملية AND بين أي bit والقيمة صفر (0): تكون النتيجة دائماً صفر [هذا يجب حجب البيانات].
- عند إجراء عملية AND بين أي bit والقيمة (1): تبقى القيمة الأصلية كما هي دون تغيير.

مثال: لنكن لدينا الرقم الثنائي:

الرقم الثنائي ⇐1011 0110 1010 0100 0011 1101 1001 1010

القناع ⇐ 0000 0000 0000 0000 0000 1111 1111 1111

- نقوم بعملية AND بين كل bit وما يقابله في القناع:

نتيجة: 0000 0000 0000 0000 0000 1101 1001 1010

- نلاحظ بالمقارنة بين النتيجة والرقم الثنائي:
  - أنه عندما قمنا بعملية AND مع 0 ← تم حجب البيانات و أصبحت أصفار
  - وعندما قمنا بعملية AND مع 1 ← تم حفظ البيانات كما هي في 12 بت الدنيا.

تصفير البتات (Zeroing bits) :

لتصفير أجزء معينة من السجل نستخدم تعيمة andi (and immediate)

Andi $t0 , $t0 , 0x0FFF

هذا القناع نلاحظ نه يحتوي على أصفار في الخانات العليا

هذا يؤدي إلى تصفير البتات المقابلة لها في السجل $t0 بشكل إجباري بينما البتات المقابلة لـ(البتات التي تحتوي F) كما هي

- ضبط البتات إلى واحد (setting some bits as ones in a word) :

لإجبار بتات معينة على أن تصبح قيمتها 1 دون تغيير بقية البتات⇐ تستخدم تعليمة ori (or immediate) وتعتمد هذه العملية على منطق (or bit – by bit)

أي بت يتم عمل or له مع 1 فتصبح نتيجته 1 حتماً

أي بت يتم عمل له or له مع 0 يبقى كما هو

مثال: لدينا السجل يحتوي القيمة $t0 = 0x12345678 لدينا العملية

هذا القناع 0xFFFF , $t0 , ori $t0

المستخدم يمثل بالثنائي 16 بت من الواحدات في الطرف الأيمن وًفار في الطرف الأيسر (16 بت العليا)

نتيجة العملية تكون $t0 = 0x1234FFFF

نلاحظ أنه 16 بت العليا (1234) لم تتغير لأنها تتم مقارنتها مع أصفار في القناع (0 مع OR) بينما الـ16 بت الدنيا (5678) من قيمة السجل قبل عملية الـOR تحولت جميعها إلى واحدات (FFFF) لأنها تم مقارنتها مع واحدات في القناع (1 مع or)

Bravoo !